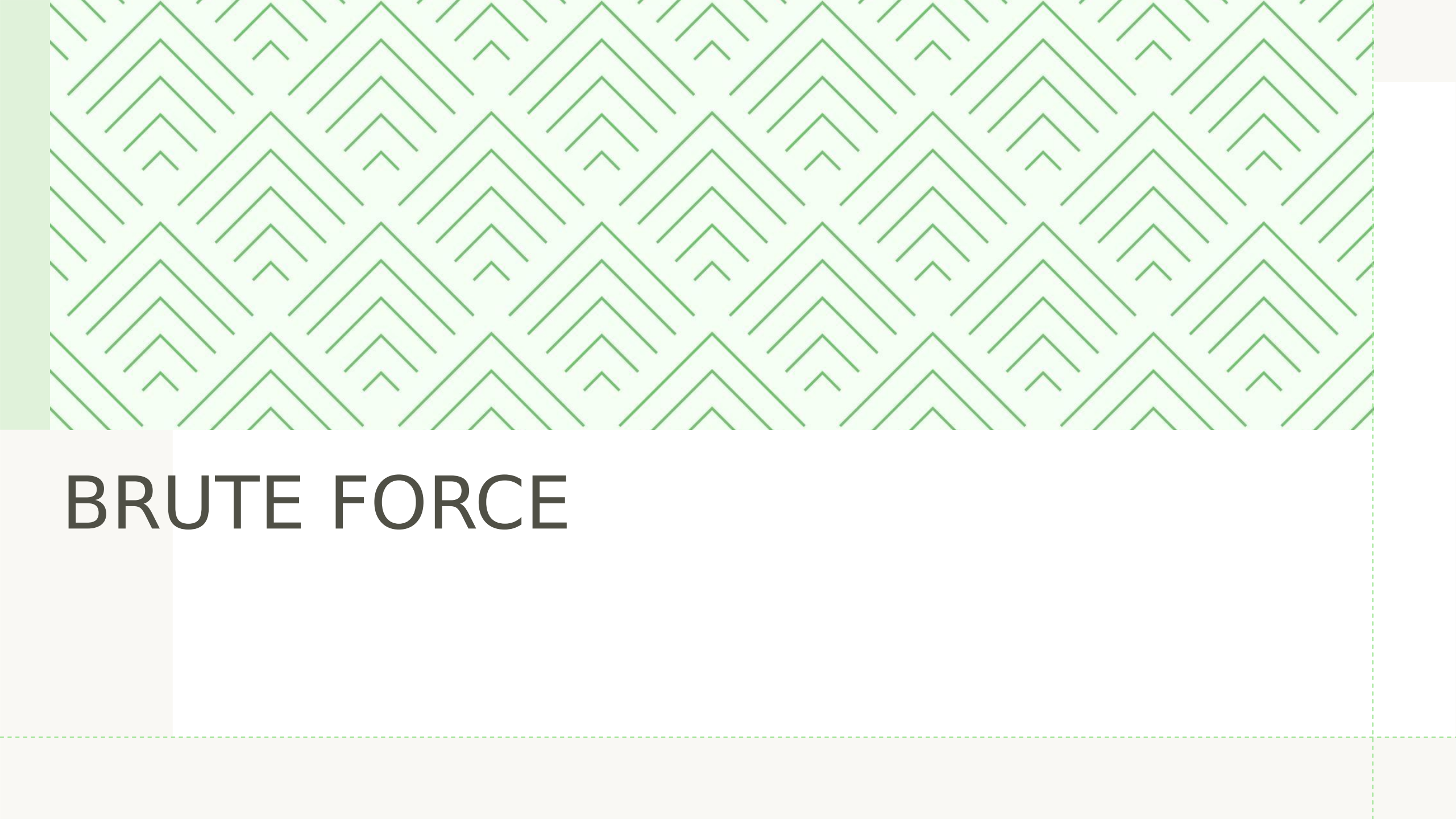


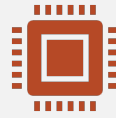


BRUTE FORCE

Brute Force



Definition: Brute force is a straightforward approach to problem-solving that relies on directly applying the problem statement and definitions of the concepts involved.



"Just do it!": The strategy's essence is to utilize the computational power of a computer rather than relying on complex intellectual reasoning.

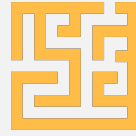


Simplicity: Brute force often represents the easiest and most intuitive method to apply when solving a problem.



Example: The exponentiation problem, where you need to compute a^b for a given number a and non-negative integer b , can be solved using brute force.

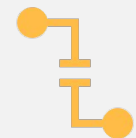
Selection Sort and Bubble Sort



The brute-force approach can also be applied to the problem of sorting, which involves rearranging a list of n orderable items in nondecreasing order.



Importance of Sorting: Sorting is a fundamental problem in computer science, and numerous algorithms have been developed to solve it efficiently.



Selection Sort: Among the two algorithms, selection sort is generally considered to be better overall and implements the brute-force approach more clearly.

Selection Sort Analysis with example

89	45	68	90	29	34	17
17	45	68	90	29	34	89
17	29	68	90	45	34	89
17	29	34	90	45	68	89
17	29	34	45	90	68	89
17	29	34	45	68	90	89
17	29	34	45	68	89	90

Selection Sort Algorithm

Key operation: Comparison & Swap

Let $C(n)$ denote total Number of Comparisons

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} (n-1-i),$$

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n-1-i) = \frac{(n-1)n}{2}.$$

Total Number of Swaps: $n - 1$

- ♦ **ALGORITHM Selection Sort($A[0 \dots n-1]$)**
- ♦ // Sorts a given array using selection sort
- ♦ // Input: An array $A[0 \dots n-1]$ of orderable elements
- ♦ // Output: Array $A[0 \dots n-1]$ sorted in ascending order
- ♦ for $i \leftarrow 0$ to $n-2$ do
 - min $\leftarrow i$
 - ♦ for $j \leftarrow i+1$ to $n-1$ do
 - ♦ if $A[j] < A[\text{min}]$
 - ♦ min $\leftarrow j$
 - ♦ swap $A[i]$ and $A[\text{min}]$

Time and Space Complexity

Time efficiency : $\Theta(n^2)$

Space efficiency : $\Theta(1)$, so in place

Stability : No

Bubble Sort Example

89	$\overset{?}{\leftrightarrow}$	45		68		90		29		34		17
45		89	$\overset{?}{\leftrightarrow}$	68		90		29		34		17
45		68		89	$\overset{?}{\leftrightarrow}$	90	$\overset{?}{\leftrightarrow}$	29		34		17
45		68		89		29		90	$\overset{?}{\leftrightarrow}$	34		17
45		68		89		29		34		90	$\overset{?}{\leftrightarrow}$	17
45		68		89		29		34		17		90
45	$\overset{?}{\leftrightarrow}$	68	$\overset{?}{\leftrightarrow}$	89	$\overset{?}{\leftrightarrow}$	29		34		17		90
45		68		29		89	$\overset{?}{\leftrightarrow}$	34		17		90
45		68		29		34		89	$\overset{?}{\leftrightarrow}$	17		90
45		68		29		34		17		89		90

etc.



Bubble Sort Algorithm

- ♦ //Sorts a given array by bubble sort in their final positions
- ♦ //Input: An array $A[0 \dots n-1]$ of orderable elements
- ♦ //Output: Array $A[0 \dots n-1]$ sorted in ascending order
- ♦ **BubbleSort($A[0 \dots n-1]$)**
 - ♦ for $i \leftarrow 0$ to $n-2$ do
 - ♦ for $j \leftarrow 0$ to $n-2-i$ do
 - ♦ if $A[j+1] < A[j]$ then
 - ♦ swap $A[j]$ and $A[j+1]$

Bubble Sort Analysis

the sum for selection sort:

$$\begin{aligned} C(n) &= \sum_{i=0}^{n-2} \sum_{j=0}^{n-2-i} 1 = \sum_{i=0}^{n-2} [(n-2-i) - 0 + 1] \\ &= \sum_{i=0}^{n-2} (n-1-i) = \frac{(n-1)n}{2} \in \Theta(n^2). \end{aligned}$$

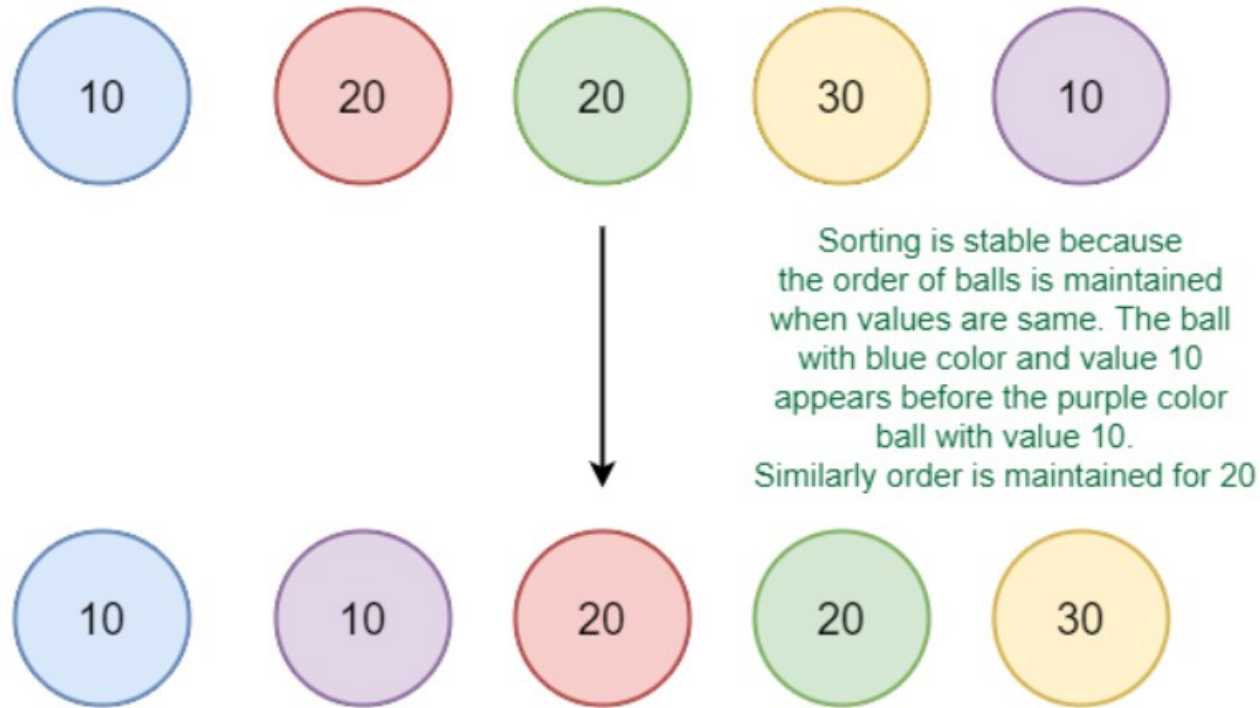
The number of key swaps, however, depends on the input. For the worst case of decreasing arrays, it is the same as the number of key comparisons:

$$S_{worst}(n) = C(n) = \frac{(n-1)n}{2} \in \Theta(n^2).$$

Stable and Unstable Sorting Algorithm

- ♦ A sorting algorithm is said to be stable if two objects with equal keys appear in the same order in sorted output as they appear in the input data set.

Informally, stability means that equivalent elements retain their relative positions, after sorting.



- Consider the following dataset of Student Names and their respective class sections.

(Dave, A)

(Alice, B)

(Carol, A)

(Dave, A)

(Alice, B)

(Carol, A)

(Dave, A)

(Ken, A)

(Ken, A)

(Dave, A)

(Ken, A)

(Carol, A)

(Eric, B)

(Eric, B)

(Eric, B)

(ALice, B)

(Carol, A)

(Ken, A)

(Alice, B)

(Eric, B)

Sort
according to
Name.
Resulting
data is not
grouped
according to
section

Sort again
section wise.
This will sort
according
section but not
according to
name

In the name-sorted
dataset, the tuple (Alice,
B) was before (Eric, B) ,
but since the sorting
algorithm is not stable, the
relative order is lost. If on
the other hand, we used a
stable sorting algorithm,

- ♦ Stable sorting algorithms: Bubble sort, Insertion sort, Merge sort, counting (rank sort)
- ♦ Unstable sorting algorithms: Selection sort, Quick sort and Heap sort.
- ♦ In general, any comparison-based sorting algorithm which is not stable by nature can be modified to be stable by changing the key comparison operation so that the comparison of two keys considers position as a factor for objects with equal keys. Selection sort can be modified as stable
- ♦ When equal elements are indistinguishable, such as with integers, or more generally, any data where the entire element is the key, stability is not an issue. Stability is also not an issue if all

Sequential Search



- ♦ //Implements sequential search with a search key as a sentinel
- ♦ //Input: An array A of n elements and a search key K
- ♦ //Output: The index of the first element in A[0 .. n -1] whose value is equal to K or -1 if no such element is found
- ♦ SequentialSearch(A[0..n], K)
 - ♦ $A[n] \leftarrow K$
 - ♦ $i \leftarrow 0$
 - ♦ while $A[i] \neq K$ do
 - ♦ $i \leftarrow i + 1$
 - ♦ if $i < n$ then
 - ♦ return i
 - ♦ else
 - ♦ return -1
- ♦ Time Complexity ---- ???

String Matching



- ♦ The string-matching problem involves finding a substring in a given text that matches a given pattern.
 - ♦ **Example : Nobody noticed him**
- ♦ In this example, the text is "Nobody noticed him" and the pattern is not specified. We'll assume the pattern is "not" for demonstration purposes.
- ♦ The goal is to find the index i of the leftmost character of the first matching substring in the text.

String Matching

N O B O D Y _ N O T I C E D _ H I M
N O N T O N T O N T O N T O N T O N T
N O N T O N T O N T O N T O N T O N T

String Matching

- ♦ The algorithm starts by comparing the first character of the pattern with the characters in the text starting from the first character.
- ♦ If there is a match, it continues to compare the next characters of the pattern with the consecutive characters in the text.
- ♦ If there is a mismatch at any point, the algorithm moves to the next character in the text and starts comparing again from there.
- ♦ In our example, the algorithm would compare "not" with the first three characters of the text - "Nob". It would find a mismatch and move to the next character.
- ♦ The algorithm continues this process until either a complete match is found or the end of the text is reached.
- ♦ If a match is found, the algorithm returns the index i of the leftmost character of the matching substring.

String Matching Algo

- ♦ **ALGORITHM Brute Force String Match($T[0..n-1], P[0..m-1]$)**
- ♦ //Implements brute-force string matching
- ♦ //Input: An array $T[0..n-1]$ of n characters representing a text and
- ♦ //an array $P[0..m-1]$ of m characters representing a pattern
- ♦ //Output: The index of the first character in the text that starts a matching substring or -1 if the search is unsuccessful for
- ♦ for $i \leftarrow 0$ to $n-m$ do
- ♦ $j \leftarrow 0$
- ♦ While $j < m$ and $P[j] = T[i+j]$ do
- ♦ $j \leftarrow j+1$
- ♦ if $j = m$ return i
- ♦ return -1

String Matching Time Complexity

- ♦ In the worst case scenario, the algorithm may have to make all m comparisons before shifting the pattern.
- ♦ This worst case can happen for each of the $n - m + 1$ tries, where n is the length of the text and m is the length of the pattern.
- ♦ So, in the worst case, the time complexity of the algorithm is $O(n * m)$, which means it grows proportionally to the product of the lengths of the text and the pattern.
- ♦ However, for a typical word search in a natural language text, most shifts happen after very few comparisons.
- ♦ This means that in practice, the average-case efficiency of the algorithm is considerably better than the worst-case efficiency.
- ♦ In fact, for searching in random texts, it has been shown that the average-case time complexity is linear, meaning it grows linearly with the sum of the lengths of the text and the pattern. So it is $O(n + m)$.
- ♦ This linear time complexity, $O(n + m)$, is much more efficient than the worst-case time complexity, $O(n * m)$.
- ♦ It's important to note that these time complexities are specific to the algorithm being used. Different string-matching algorithms may have different time complexities and performance characteristics.

Exhaustive search



Exhaustive search is a method in which solution is obtained by searching each element of given problem.

Exhaustive search is a brute force method for combinatorial optimization problems.



Travelling salesman problem



Knapsack problem



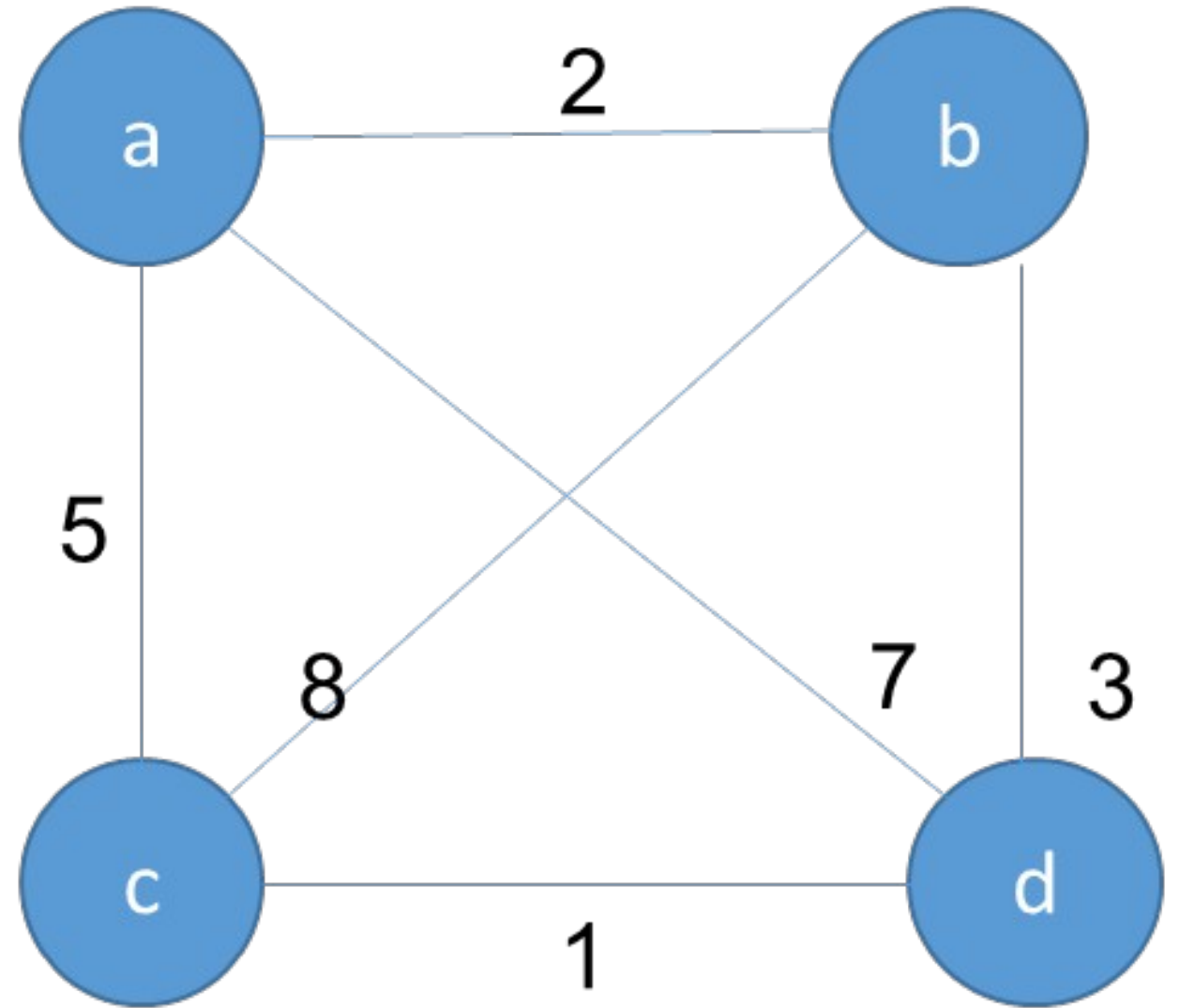
Assignment problem



Traveling Salesman Problem (TSP)

- ♦ The TSP involves finding the shortest possible route that a salesman can take to visit a set of cities and return to the starting city, while visiting each city exactly once.
- ♦ The cities and their connections are represented as a weighted graph, where the weights along the edges represent the distances or costs between the cities. The goal is to find the route with the minimum total weight.

TSP PROBLEM



TSP - SOLUTION

Tour	Length	Solution
a-b-c-d-a	$2+8+1+7=18$	
a-b-d-c-a	$2+3+1+5=11$	Optimal solution
a-c-b-d-a	$5+8+3+7=23$	
a-c-d-b-a	$5+1+3+2=11$	Optimal solution
a-d-b-c-a	$7+3+8+5=23$	
a-d-c-b-a	$7+1+8+2=18$	

Efficiency: $(n-1)!$

KNAPSACK PROBLEM

- ♦ The Knapsack Problem involves selecting a subset of items from a given set, where each item has a weight and a benefit value. The goal is to maximize the total benefit value of the selected items while ensuring that the total weight does not exceed the knapsack's maximum capacity.
- ♦ The problem can be formulated as an optimization problem, where the decision variables represent whether an item is included in the knapsack or not. Different algorithms, such as dynamic programming or greedy algorithms, can be used to solve the Knapsack Problem and find the optimal solution.

Item i	Weight W_i	Value V_i
1	2	\$1
2	3	\$2
3	4	\$8
4	5	\$6

KNAPSACK PROBLEM

The capacity of the Knapsack = 8

KNAPSACK SOLUTION

ITEM I	WEIGHT w_i	VALUE v_i	
Item 1	$\sum w_i = 2$	$\sum V_i = \$1$	
Item 2	$\sum w_i = 3$	$\sum V_i = \$2$	
Item 3	$\sum w_i = 4$	$\sum V_i = \$8$	
Item 4	$\sum w_i = 5$	$\sum V_i = \$6$	
Item 1,2	$\sum w_i = 2+3=5$	$\sum V_i = 1+2=\$3$	
Item 1,3	$\sum w_i = 2+4=6$	$\sum V_i = 1+8=\$9$	
Item 1,4	$\sum w_i = 2+5=7$	$\sum V_i = 1+6=\$7$	
Item 2,3	$\sum w_i = 3+4=7$	$\sum V_i = 2+8=\$10$	Feasible soln
Item 2,4	$\sum w_i = 3+5=8$	$\sum V_i = 2+6=\$8$	
Item 3,4	$\sum w_i = 4+5=9$	$\sum V_i = 8+6=\$14$	Not feasible
Item 1,2,3	$\sum w_i = 2+3+4=9$	$\sum V_i = 1+2+8=\$11$	Not feasible
Item 1,2,4	$\sum w_i = 2+3+5=10$	$\sum V_i = 1+2+6=\$9$	Not feasible
Item 2,3,4	$\sum w_i = 3+4+5=12$	$\sum V_i = 2+8+6=\$16$	Not feasible
Item 1,2,3,4	$\sum w_i = 2+3+4+5=14$	$\sum V_i = 1+2+8+6=\$17$	Not feasible

Assignment Problem

- ♦ The Assignment Problem involves assigning n people to n jobs, with the goal of minimizing the total cost associated with the assignments. Each person can be assigned to only one job, and each job can be assigned to only one person.
- ♦ The cost of assigning a particular person to a particular job is represented by the cost matrix $C[i, j]$. The objective is to find the assignment that results in the minimum total cost across all assignments.

Assignment Problem

	Job 1	job 2	Job 3	job 4
Person 1	10	3	8	9
Person 2	7	5	4	8
Person 3	6	9	2	9
Person 4	8	7	10	5

Assignment Problem- Solution

<1,2,3,4>

cost=10+5+2+5=22

<1,2,4,3>

cost=10+5+9+10=34

<1,3,2,4>

cost=10+4+9+5=28

<1,3,4,2>

cost=10+4+9+7=30

<1,4,2,3>

cost=10+8+9+10=37

<1,4,3,2>

cost=10+8+2+7=27

Assignment Problem- Solution

$\langle 2, 1, 3, 4 \rangle$

$\text{cost} = 3 + 7 + 2 + 5 = 17$

$\langle 2, 1, 4, 3 \rangle$

$\text{cost} = 3 + 7 + 9 + 10 = 29$

$\langle 2, 3, 4, 1 \rangle$

$\text{cost} = 3 + 4 + 9 + 8 = 24$

$\langle 2, 3, 1, 4 \rangle$

$\text{cost} = 3 + 4 + 6 + 5 = 18$

$\langle 2, 4, 1, 3 \rangle$

$\text{cost} = 3 + 8 + 6 + 10 = 27$

$\langle 2, 4, 3, 1 \rangle$

$\text{cost} = 3 + 8 + 2 + 8 = 21$

Assignment Problem- Solution

$\langle 3, 1, 2, 4 \rangle$

$\text{cost} = 8 + 7 + 9 + 5 = 29$

$\langle 3, 1, 4, 2 \rangle$

$\text{cost} = 8 + 7 + 9 + 8 = 32$

$\langle 3, 2, 4, 1 \rangle$

$\text{cost} = 8 + 5 + 9 + 8 = 30$

$\langle 3, 2, 1, 4 \rangle$

$\text{cost} = 8 + 5 + 6 + 5 = 24$

$\langle 3, 4, 1, 2 \rangle$

$\text{cost} = 8 + 8 + 6 + 7 = 29$

$\langle 3, 4, 2, 1 \rangle$

$\text{cost} = 8 + 8 + 9 + 8 = 33$

Assignment Problem- Solution

$\langle 4, 1, 2, 3 \rangle$

$\text{cost} = 9 + 7 + 9 + 10 = 35$

$\langle 4, 1, 3, 2 \rangle$

$\langle 4, 2, 1, 3 \rangle$

$\langle 4, 2, 3, 1 \rangle$

$\langle 4, 3, 1, 2 \rangle$

$\langle 4, 3, 2, 1 \rangle$

Assignment Problem Analysis

Thus by trying 24 permutations ($n! = 4! = 24$), we can obtain optimum solution. The optimum solution is $\langle 2, 1, 3, 4 \rangle$ i.e. $\text{cost} = 3 + 7 + 2 + 5 = 17$.

Thus we have to generate $n!$ instances to find solution using exhaustive search. This also shows how inefficient exhaustive search method is for solving such problems.

For solving these types of problems, many efficient algorithms are available.

DFS & BFS

■ Traversal

Given $G=(V,E)$ and vertex v , find all $w \in V$, such that w connects v .

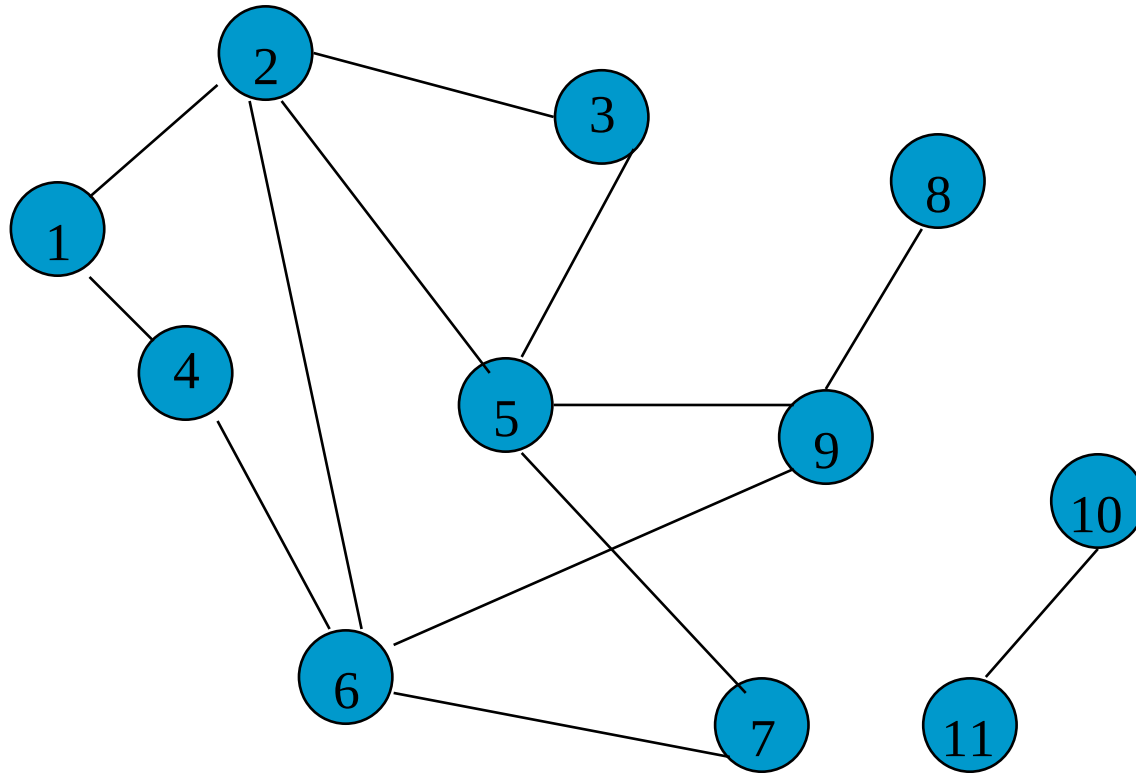
- Depth First Search (DFS)
preorder tree traversal
- Breadth First Search (BFS)
level order tree traversal

■ Applications:

- To find Connected Components of a graph G
- Spanning Tree : A subgraph T of G is said to be spanning tree of the graph G if T is a tree and consist all the vertices of G
- Find path between two vertices in a graph

Breadth First Search

- Method of starting at a vertex and identifying all vertices reachable from it.
 - Implemented using queue



Pseudocode for BFS

BFS(v)

{ Label vertex v as reached.

Initialize Q queue with only v in it

while(Q is not empty)

{

delete a vertex w from the queue;

Let u be any vertex adjacent from w;

while(u

{

if(u has not been labeled){

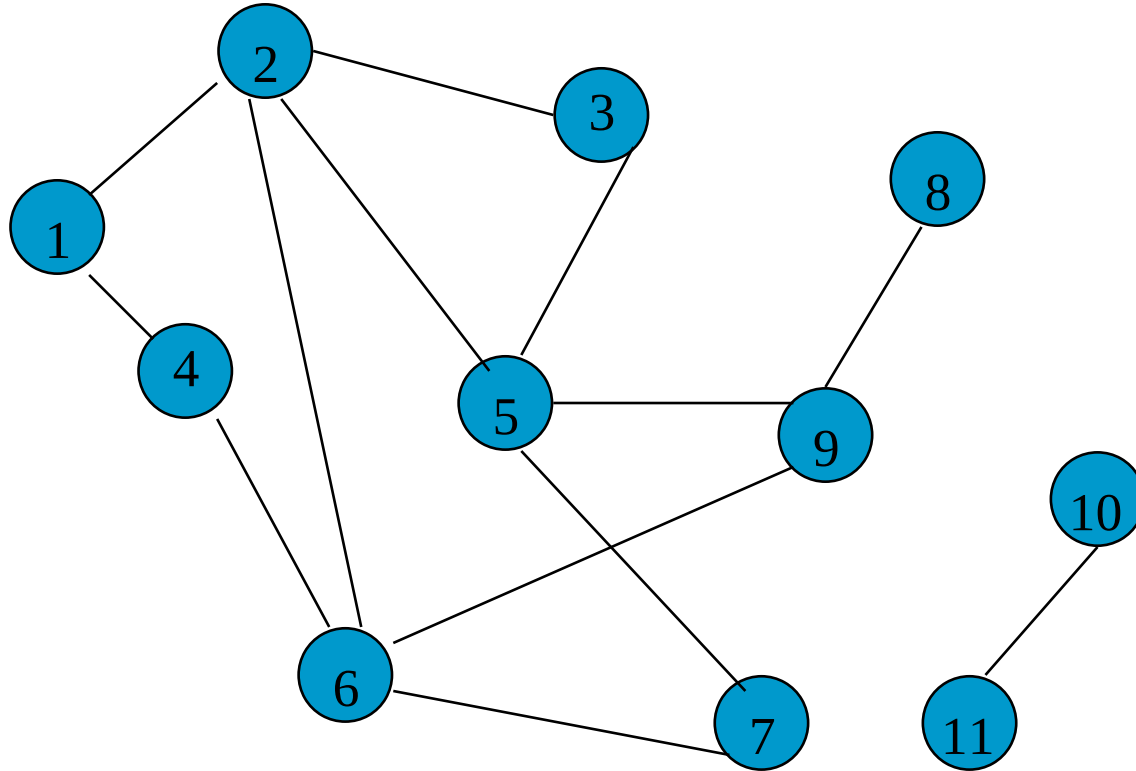
Add u to queue;

label u as reached; }

u=next vertex adjacent from w;

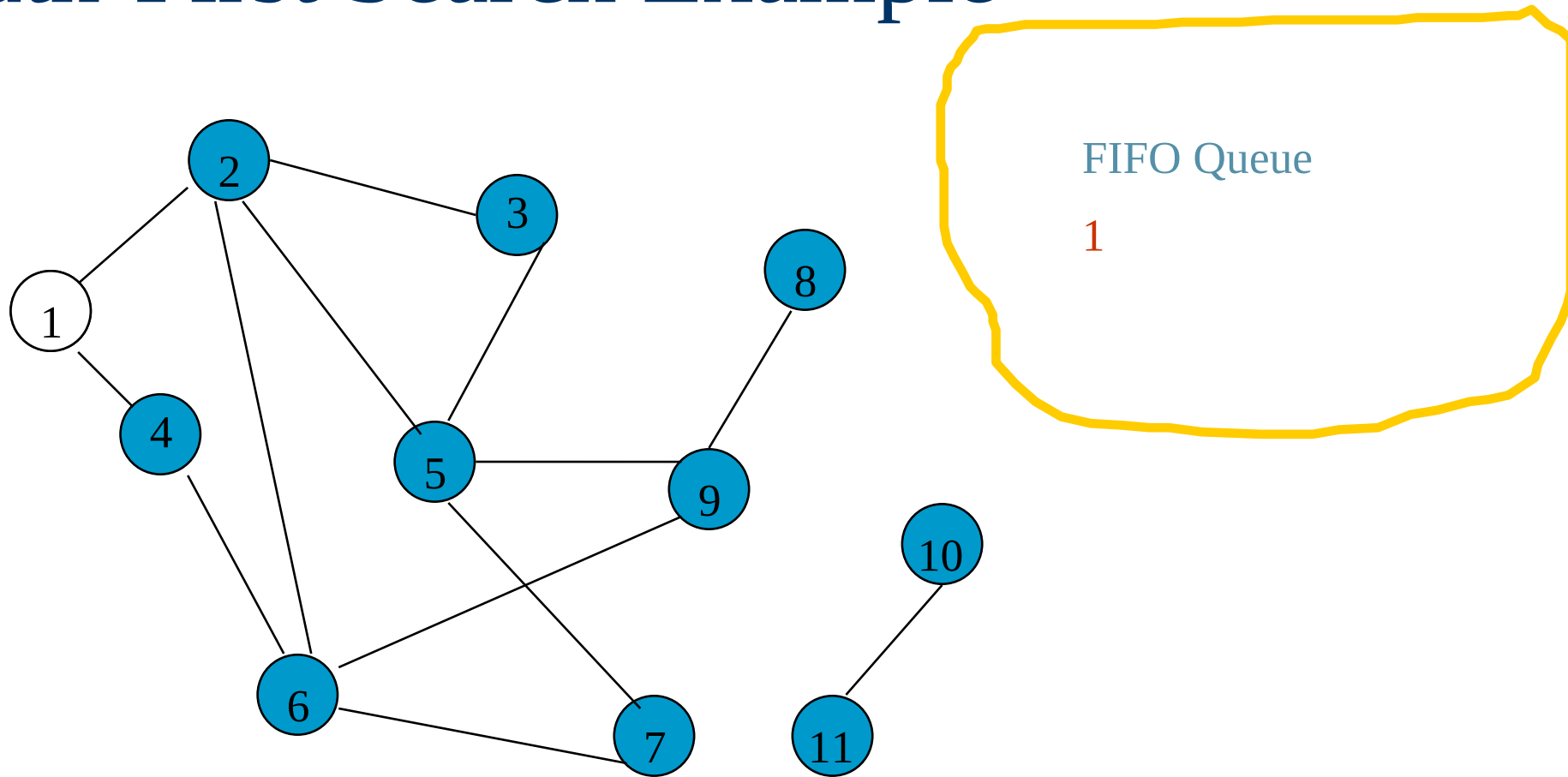
}}

Breadth-First Search Example



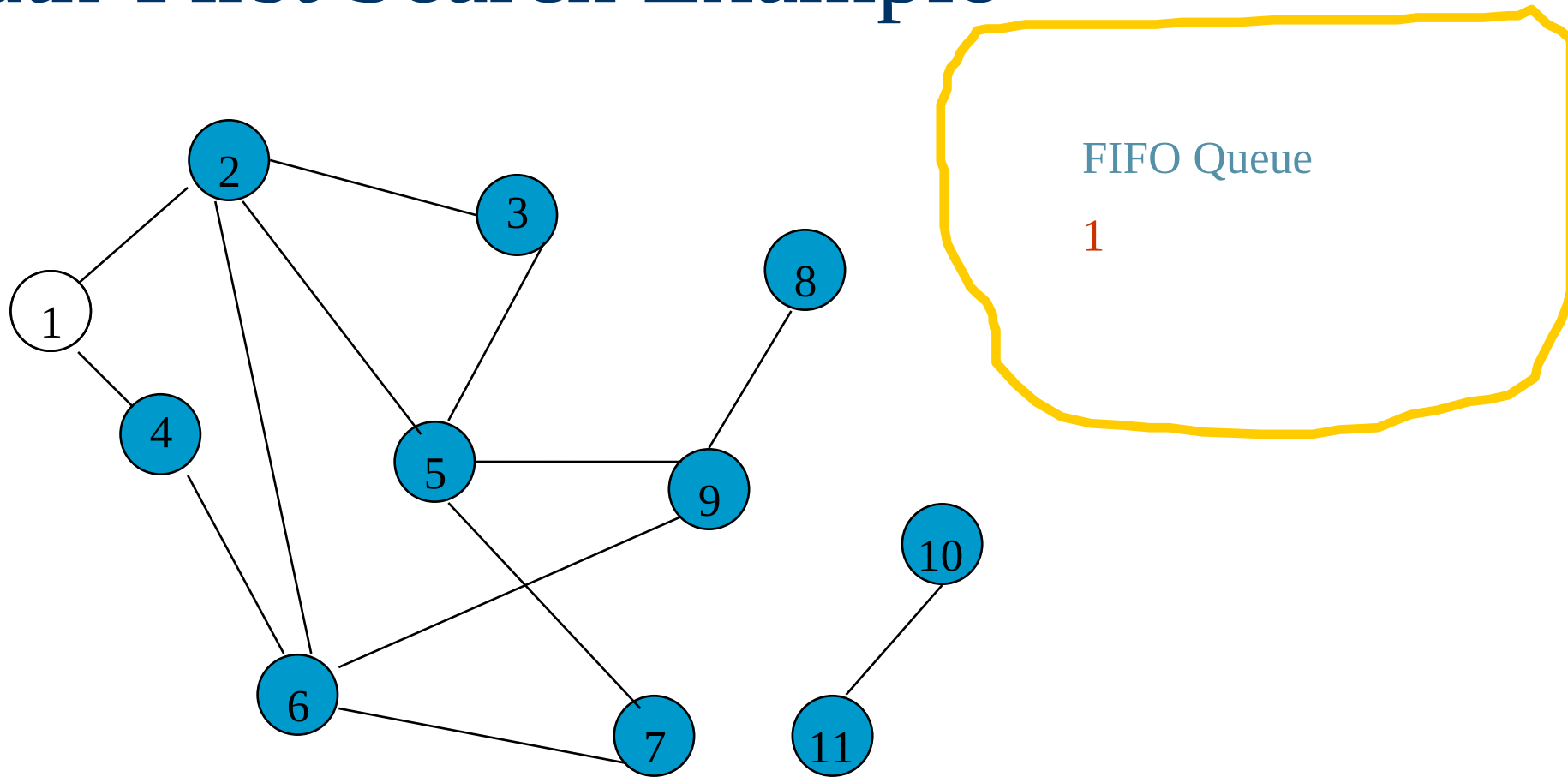
Start search at vertex 1.

Breadth-First Search Example



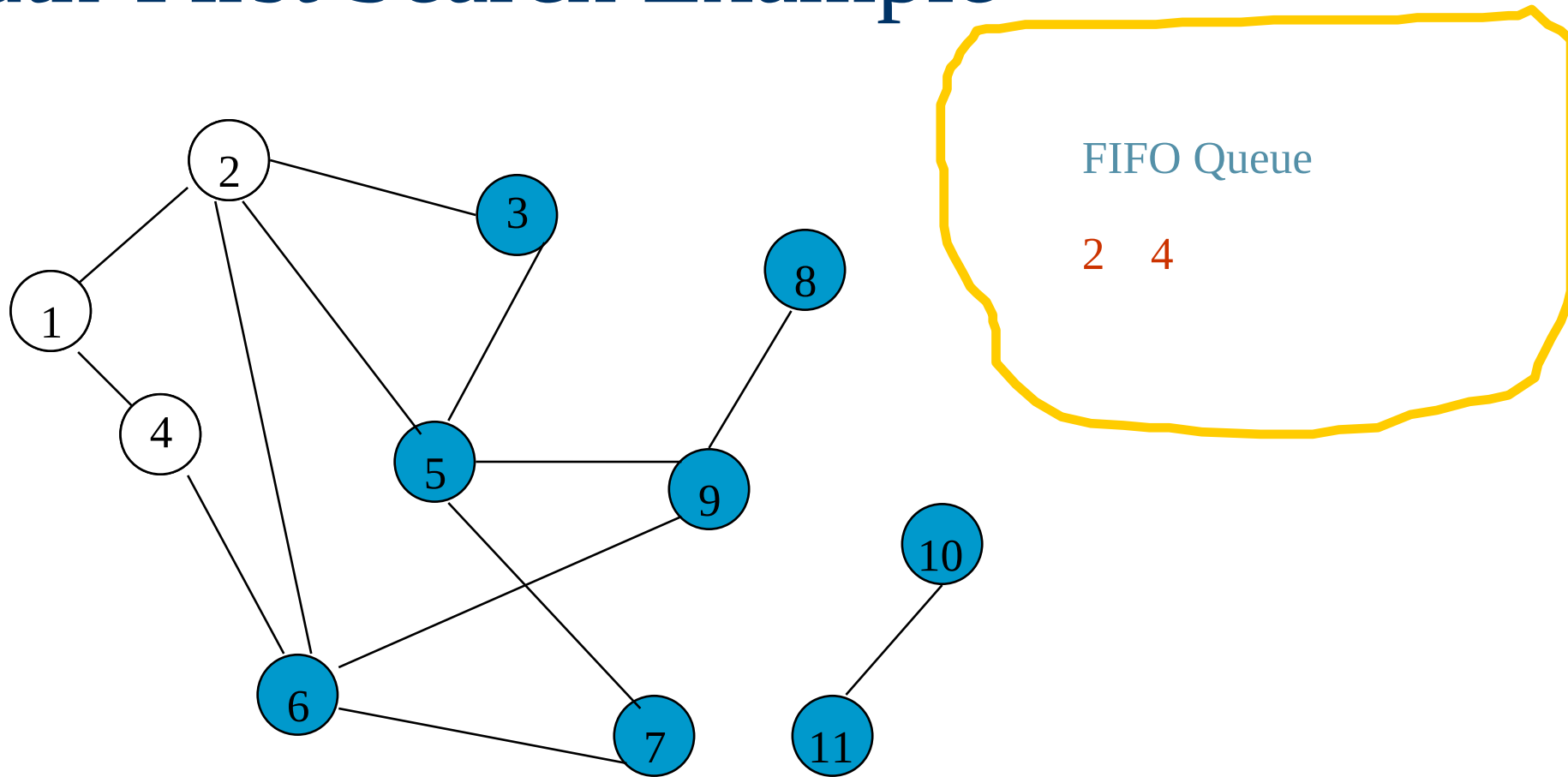
Visit/mark/label start vertex and put in a FIFO queue.

Breadth-First Search Example



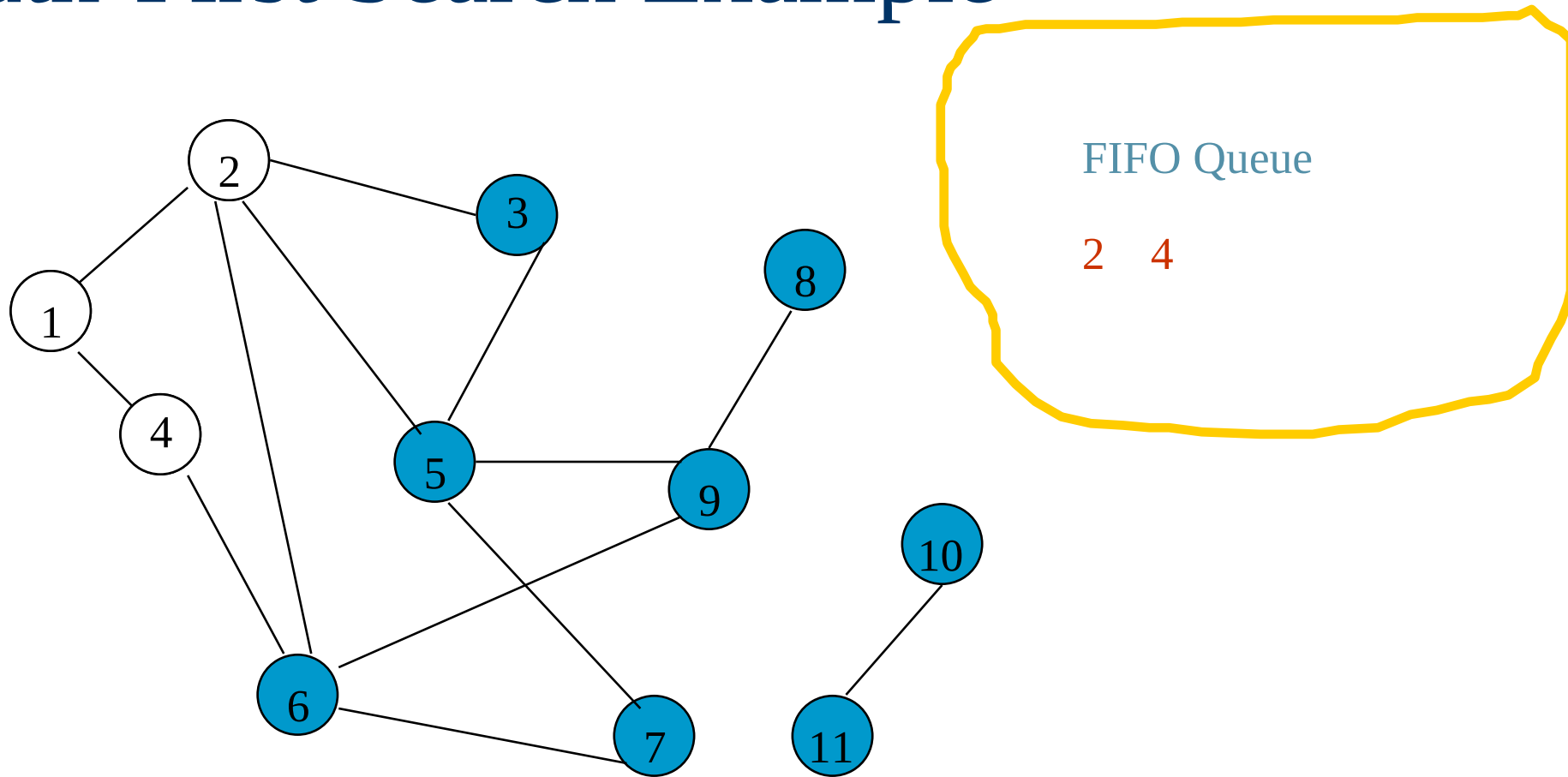
Remove 1 from Q; visit adjacent unvisited vertices;
put in Q.

Breadth-First Search Example



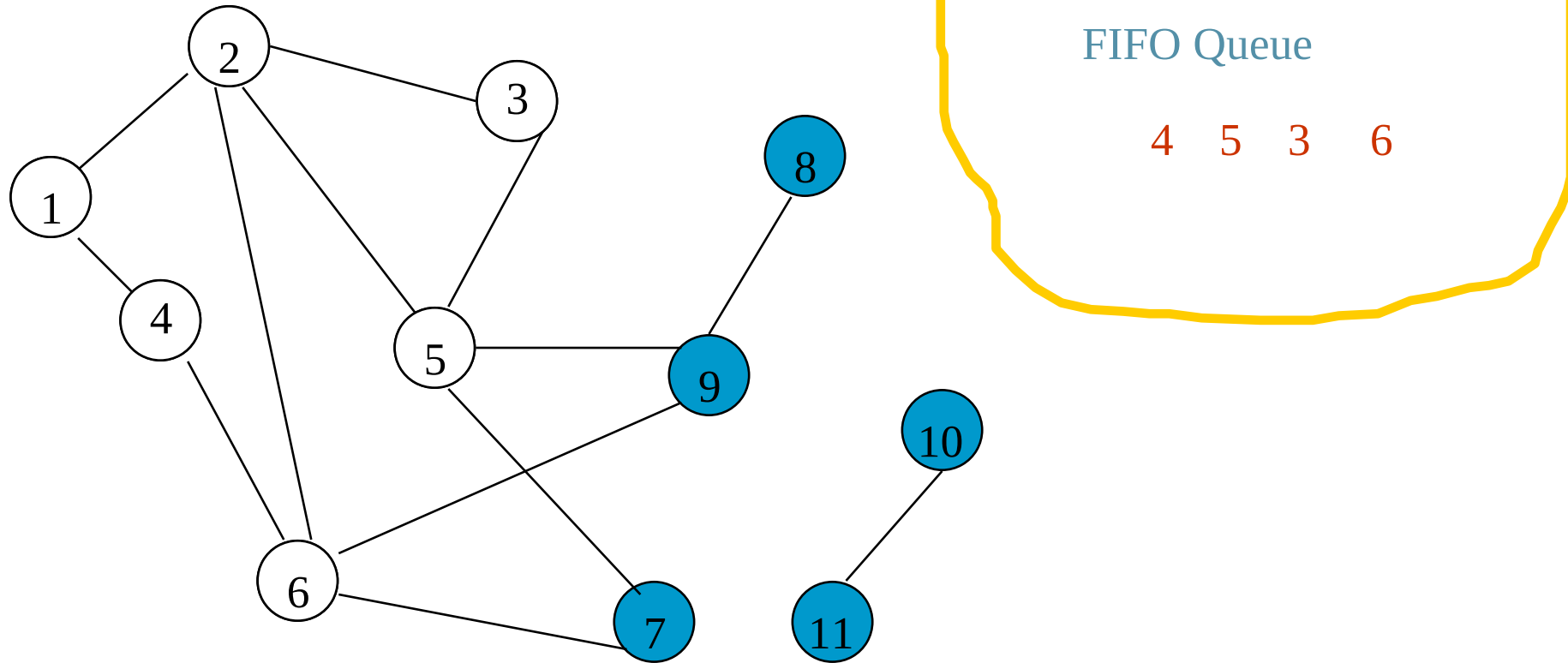
Remove 1 from Q; visit adjacent unvisited vertices;
put in Q.

Breadth-First Search Example



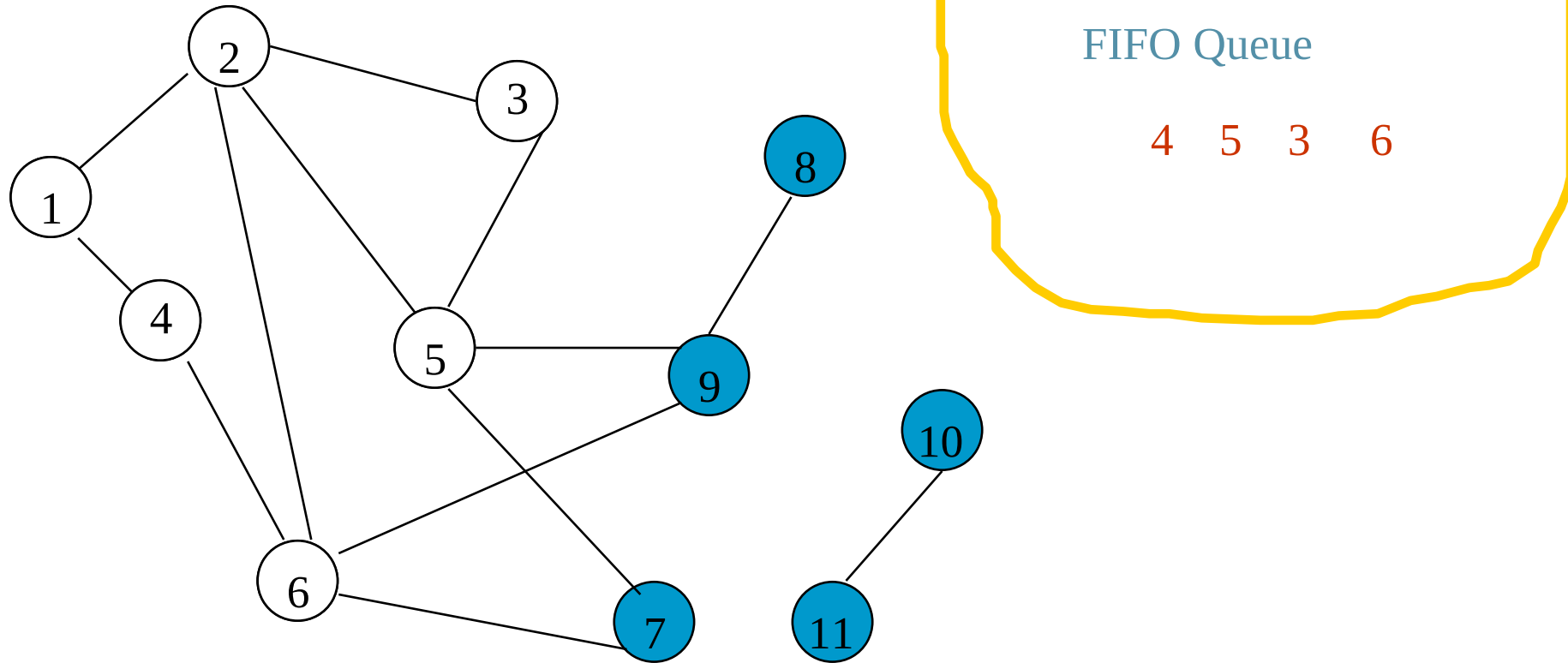
Remove 2 from Q; visit adjacent unvisited vertices;
put in Q.

Breadth-First Search Example



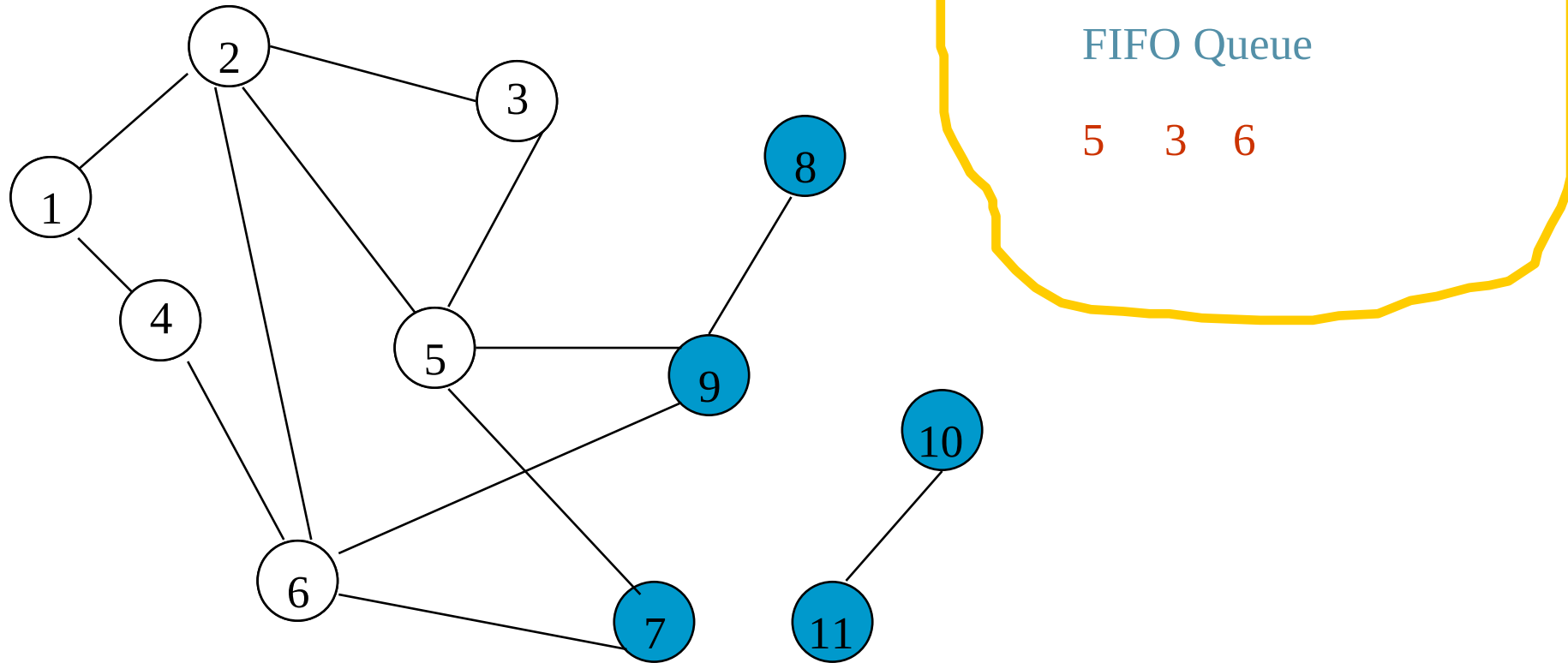
Remove 2 from Q; visit adjacent unvisited vertices;
put in Q.

Breadth-First Search Example



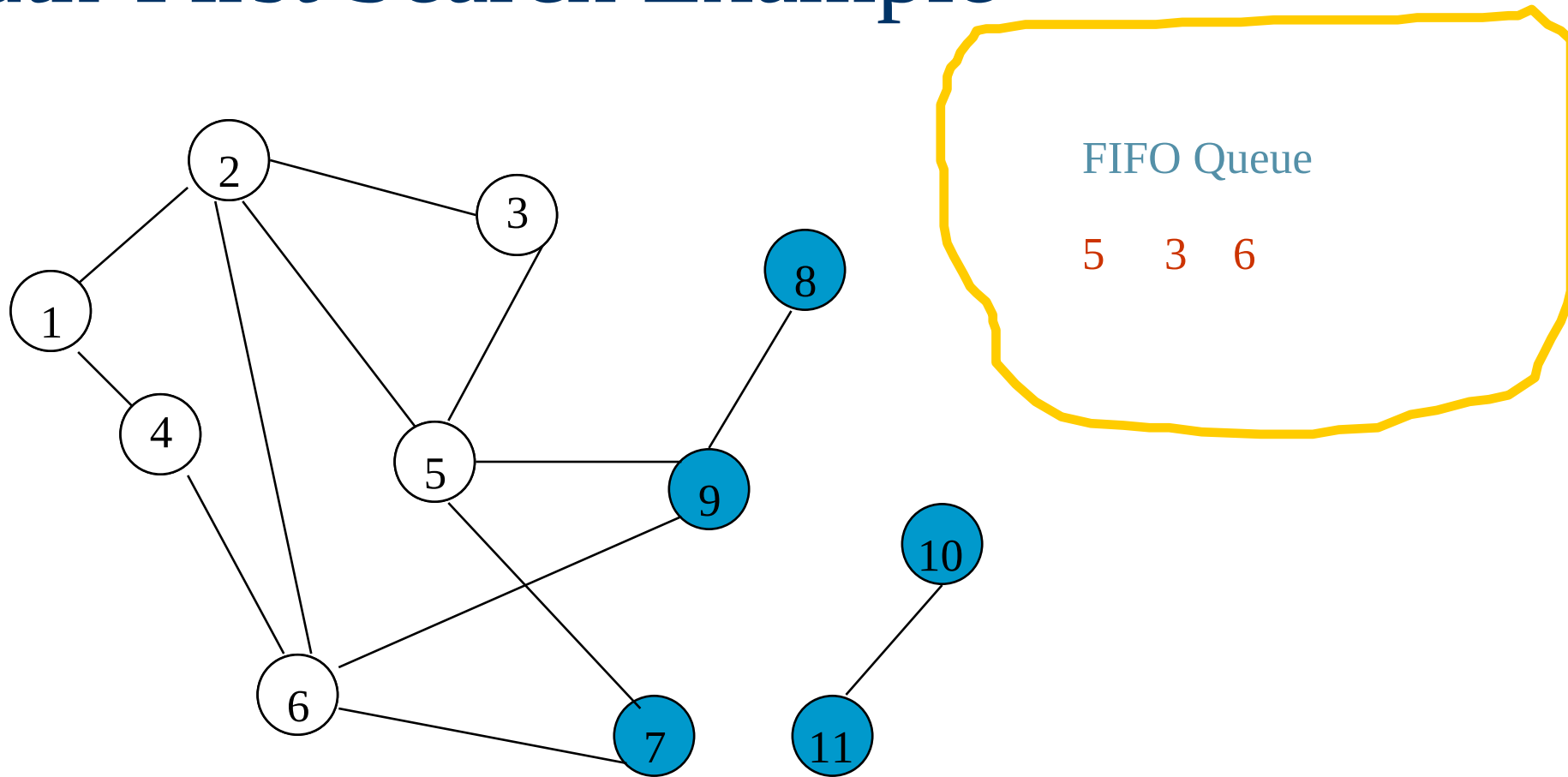
Remove 4 from Q; visit adjacent unvisited vertices;
put in Q.

Breadth-First Search Example



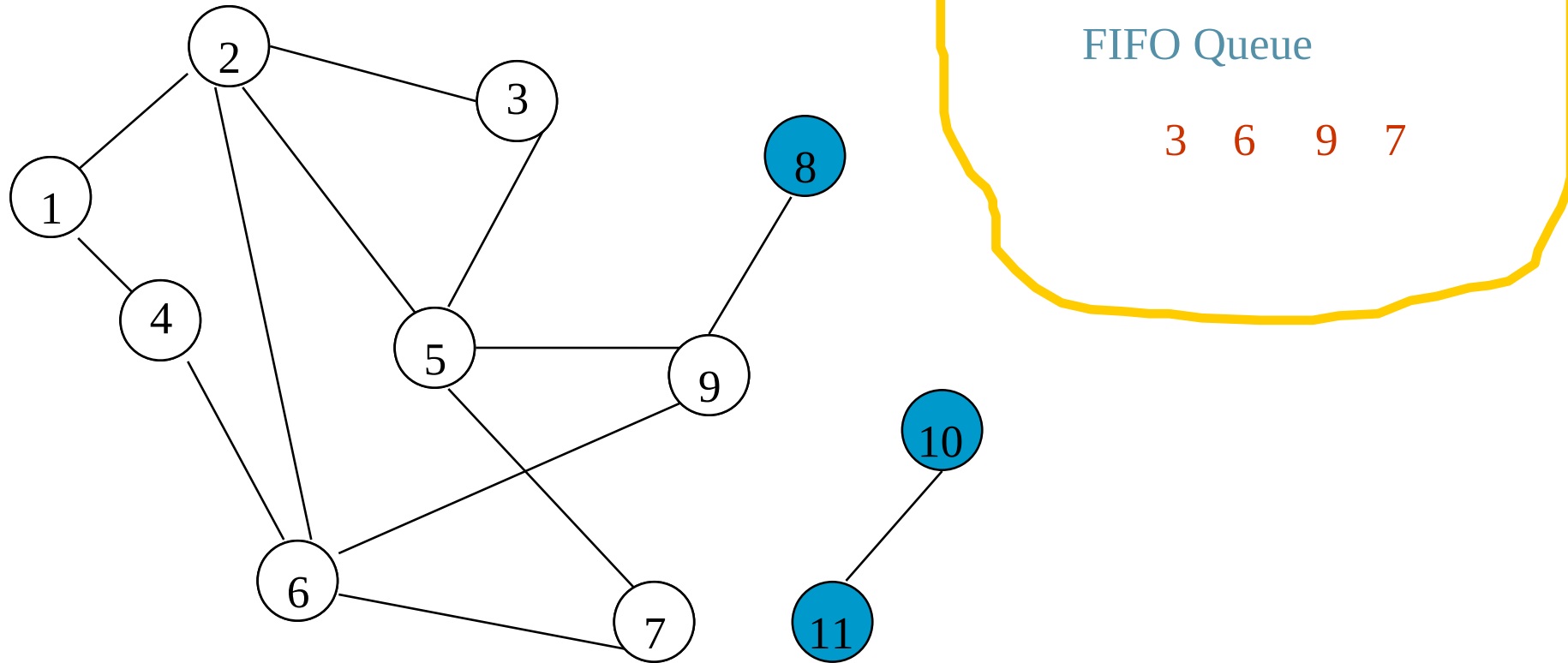
Remove 4 from Q; visit adjacent unvisited vertices;
put in Q.

Breadth-First Search Example



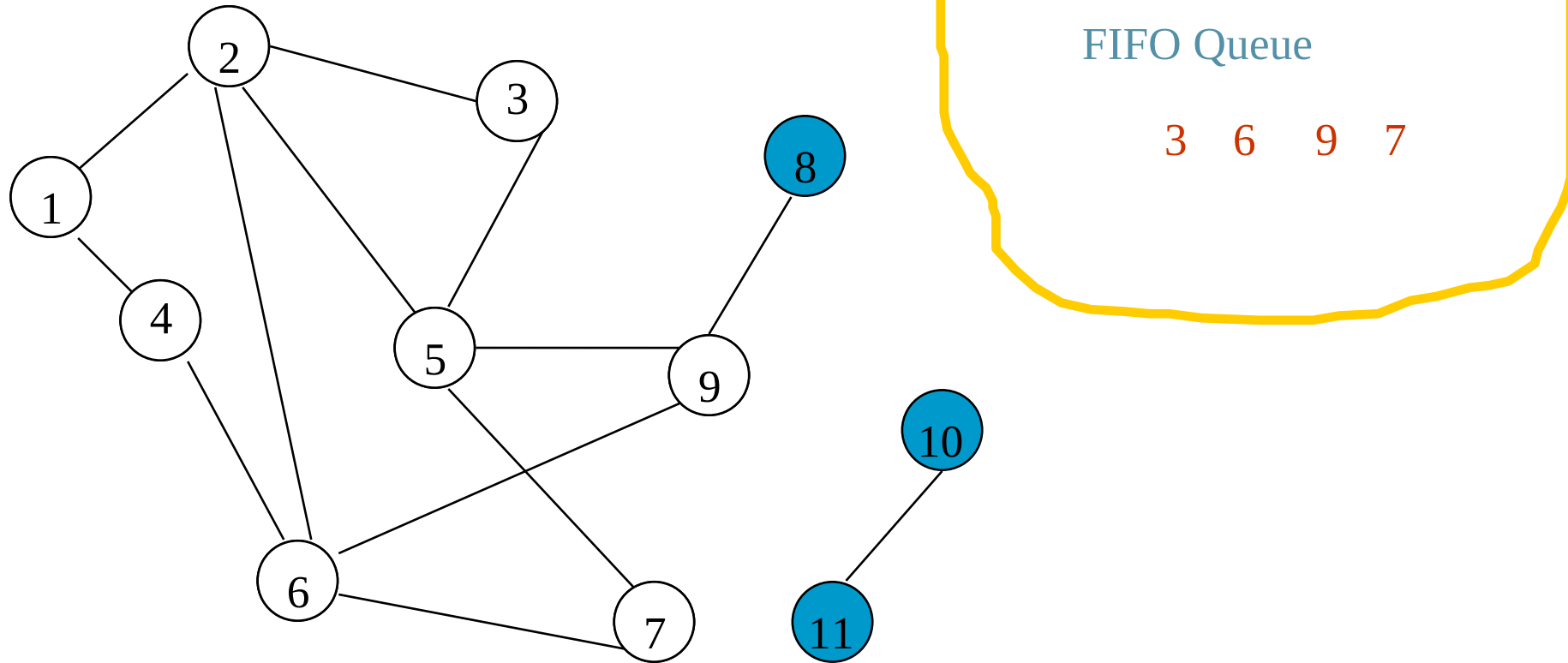
Remove 5 from Q; visit adjacent unvisited vertices;
put in Q.

Breadth-First Search Example



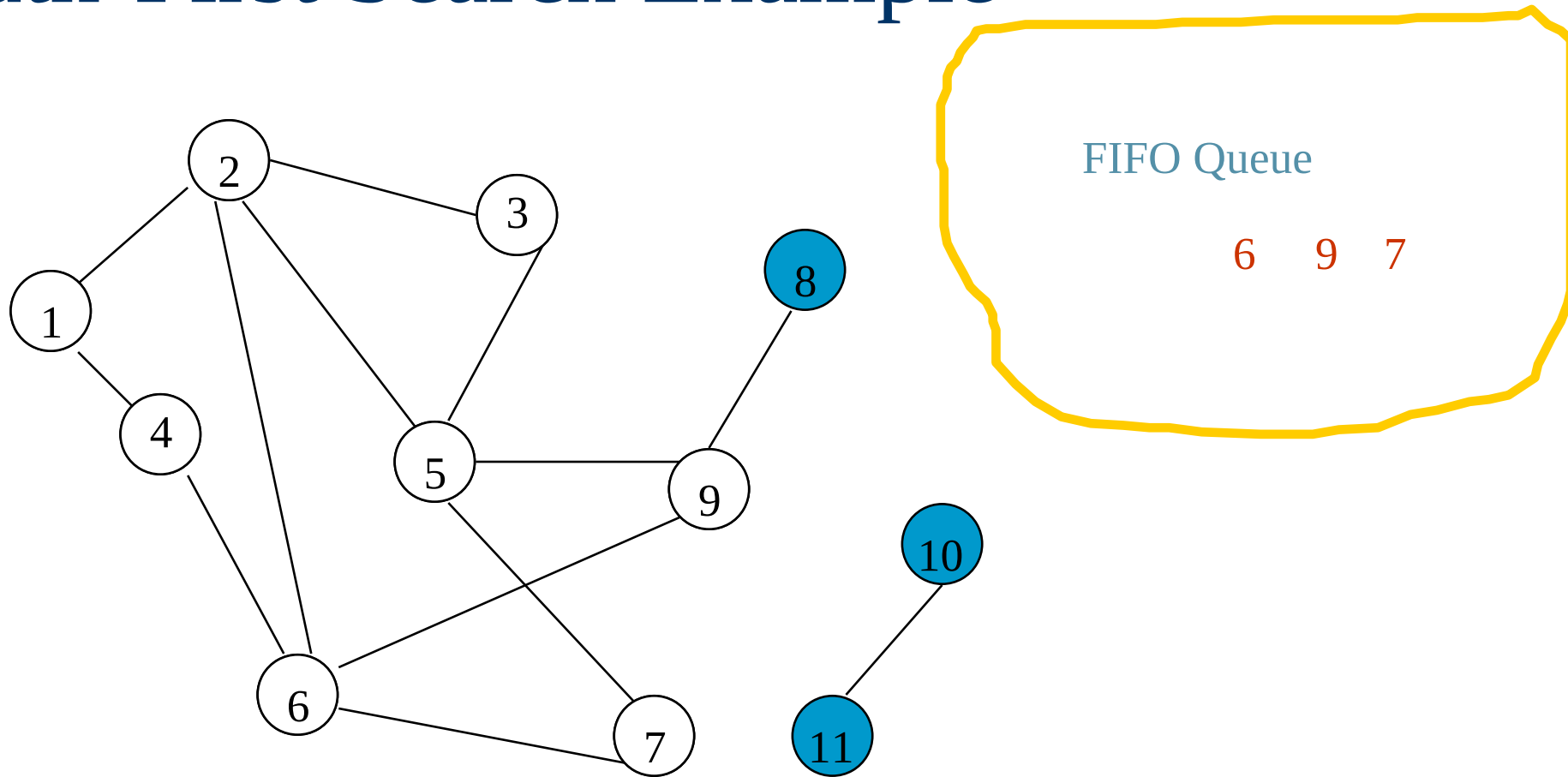
Remove 5 from Q; visit adjacent unvisited vertices;
put in Q.

Breadth-First Search Example



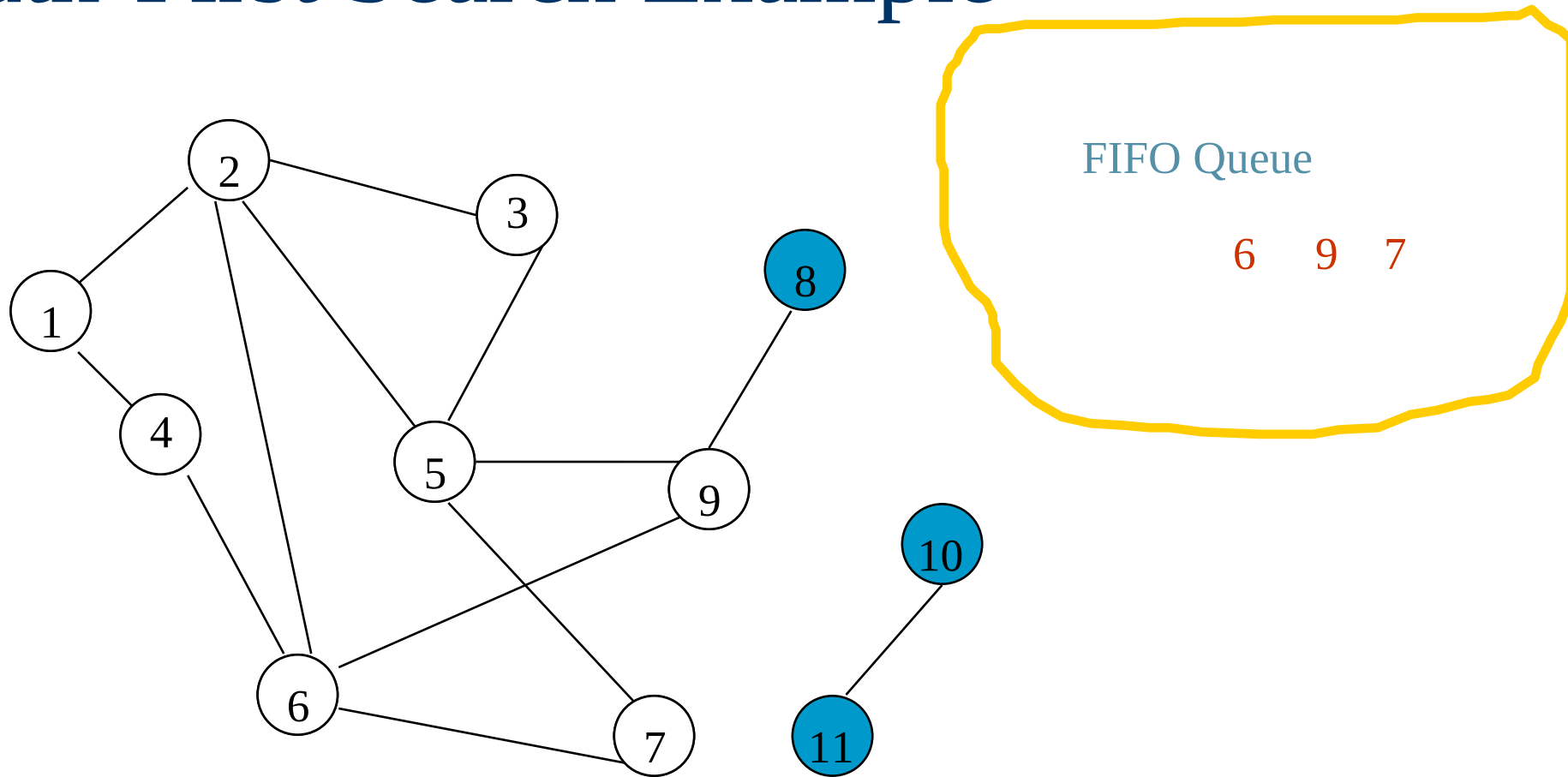
Remove 3 from Q; visit adjacent unvisited vertices;
put in Q.

Breadth-First Search Example



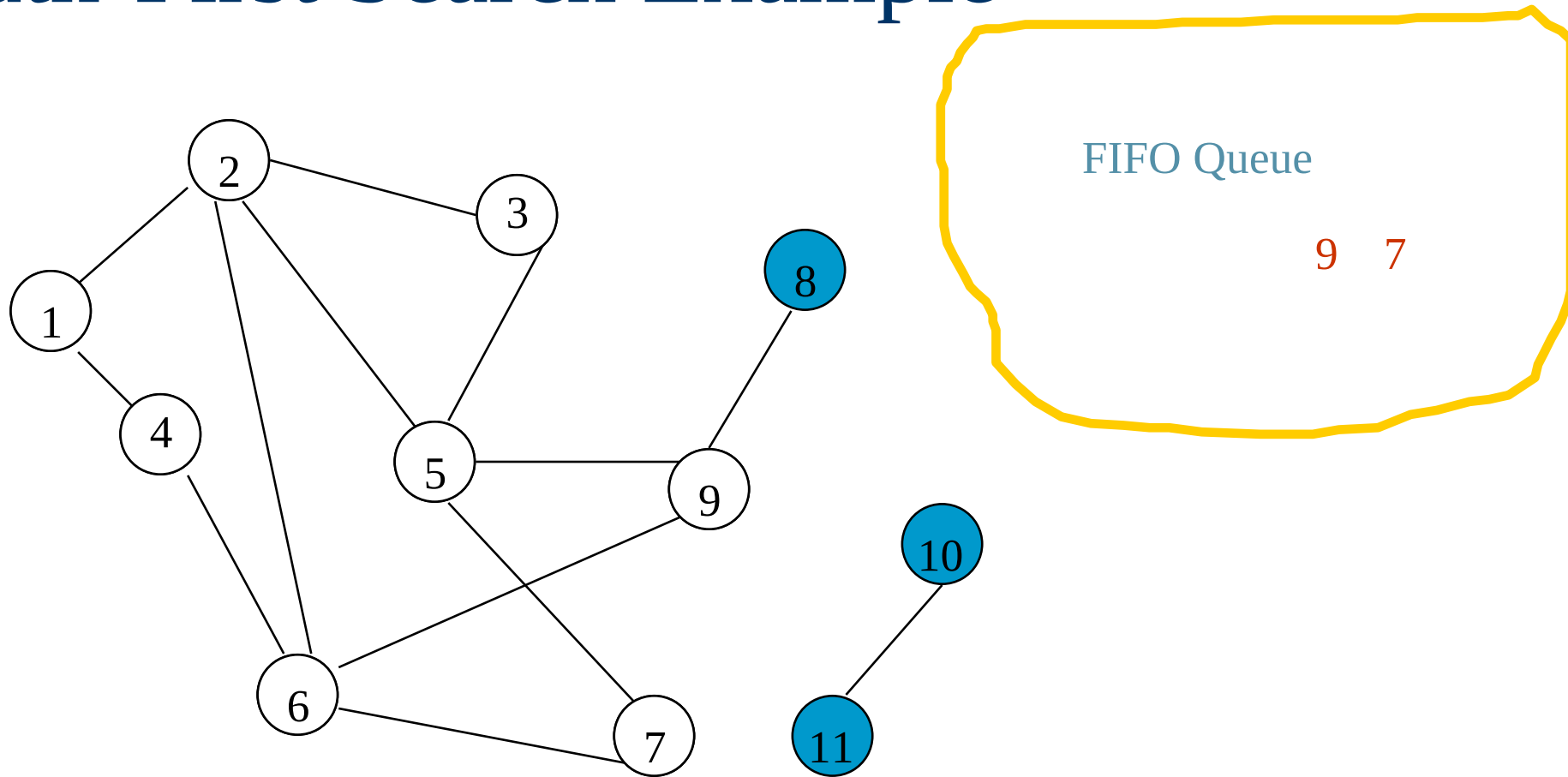
Remove 3 from Q; visit adjacent unvisited vertices;
put in Q.

Breadth-First Search Example



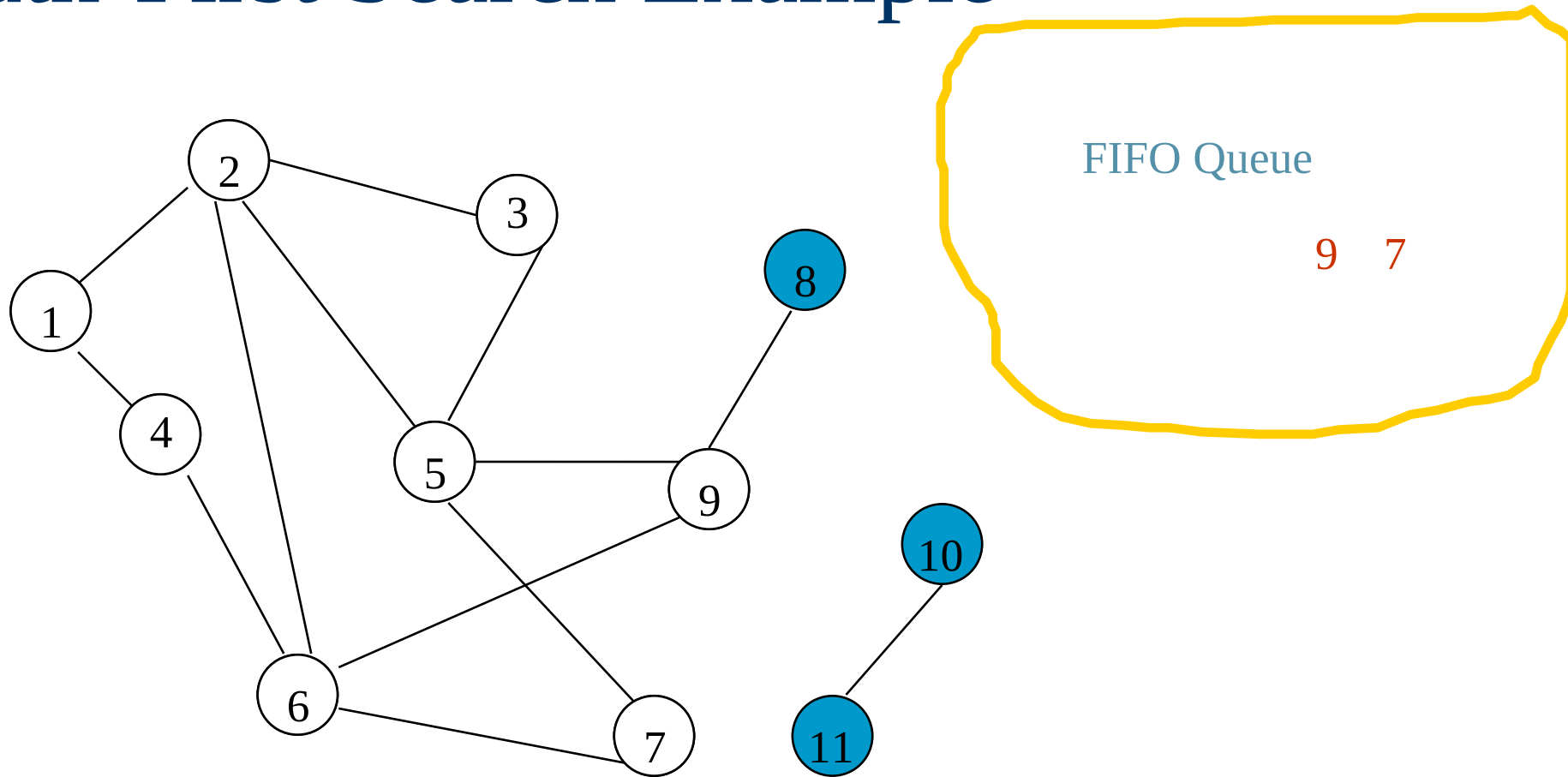
Remove 6 from Q; visit adjacent unvisited vertices;
put in Q.

Breadth-First Search Example



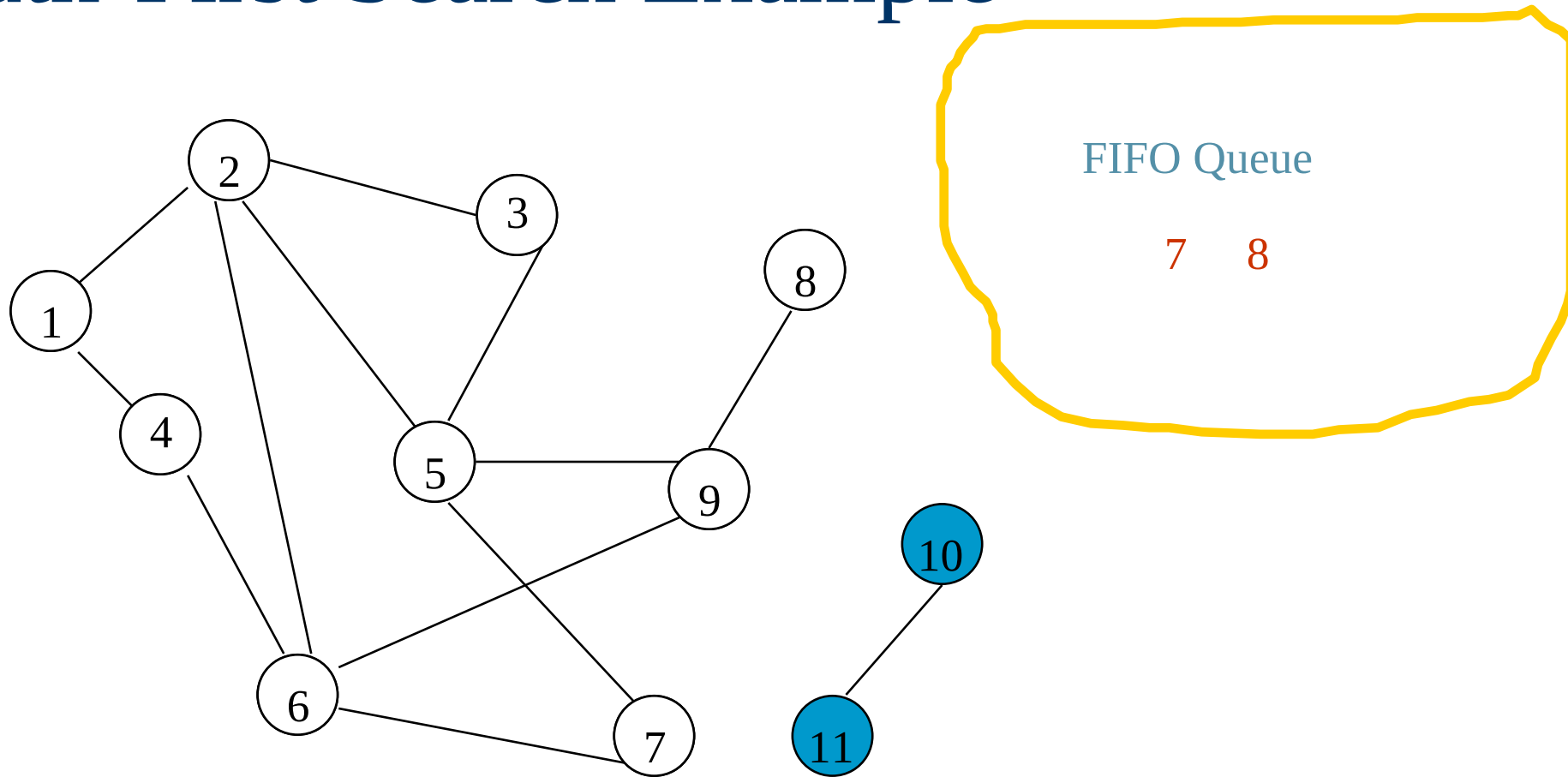
Remove 6 from Q; visit adjacent unvisited vertices;
put in Q.

Breadth-First Search Example



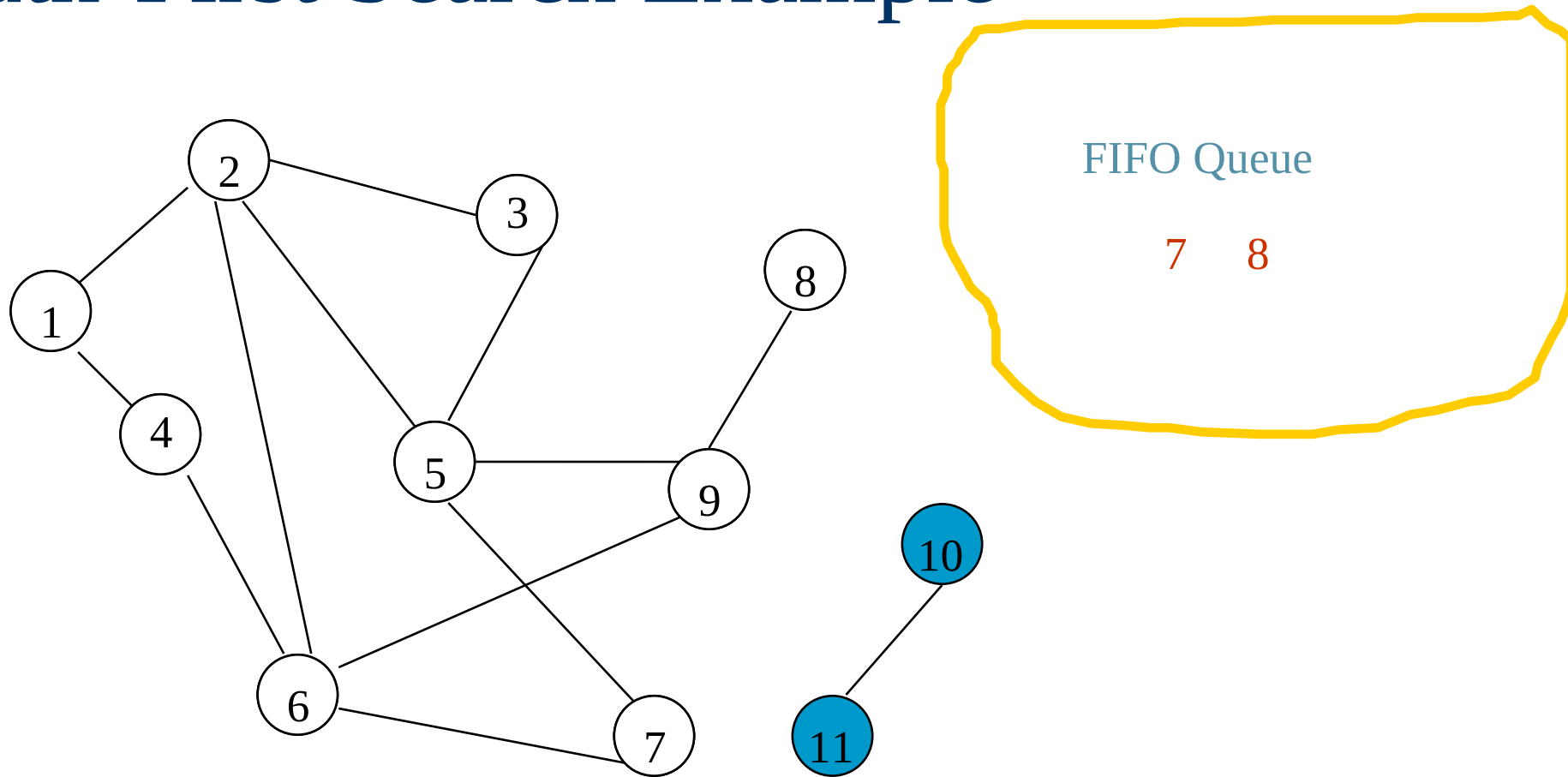
Remove 9 from Q; visit adjacent unvisited vertices;
put in Q.

Breadth-First Search Example



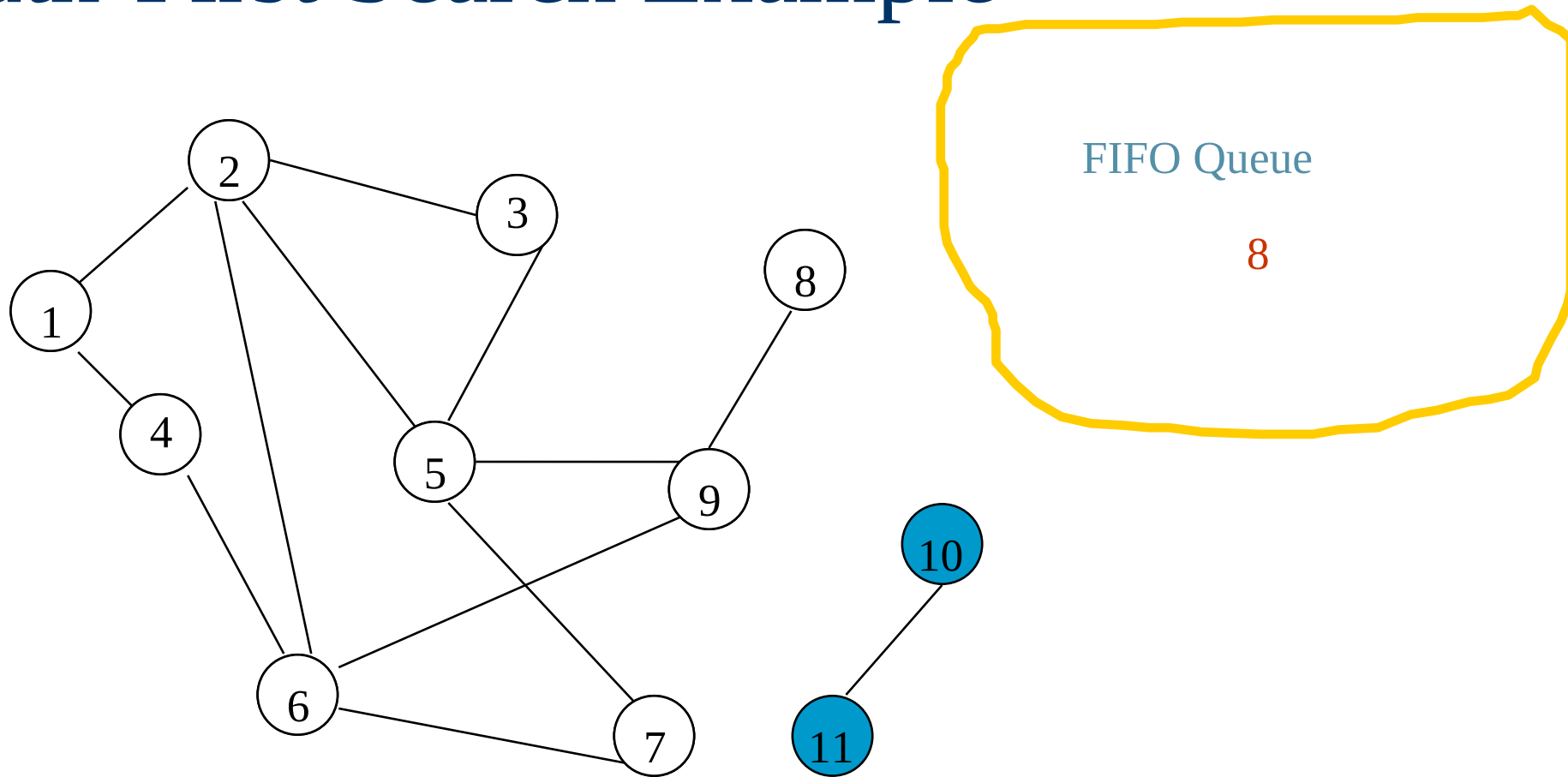
Remove 9 from Q; visit adjacent unvisited vertices;
put in Q.

Breadth-First Search Example



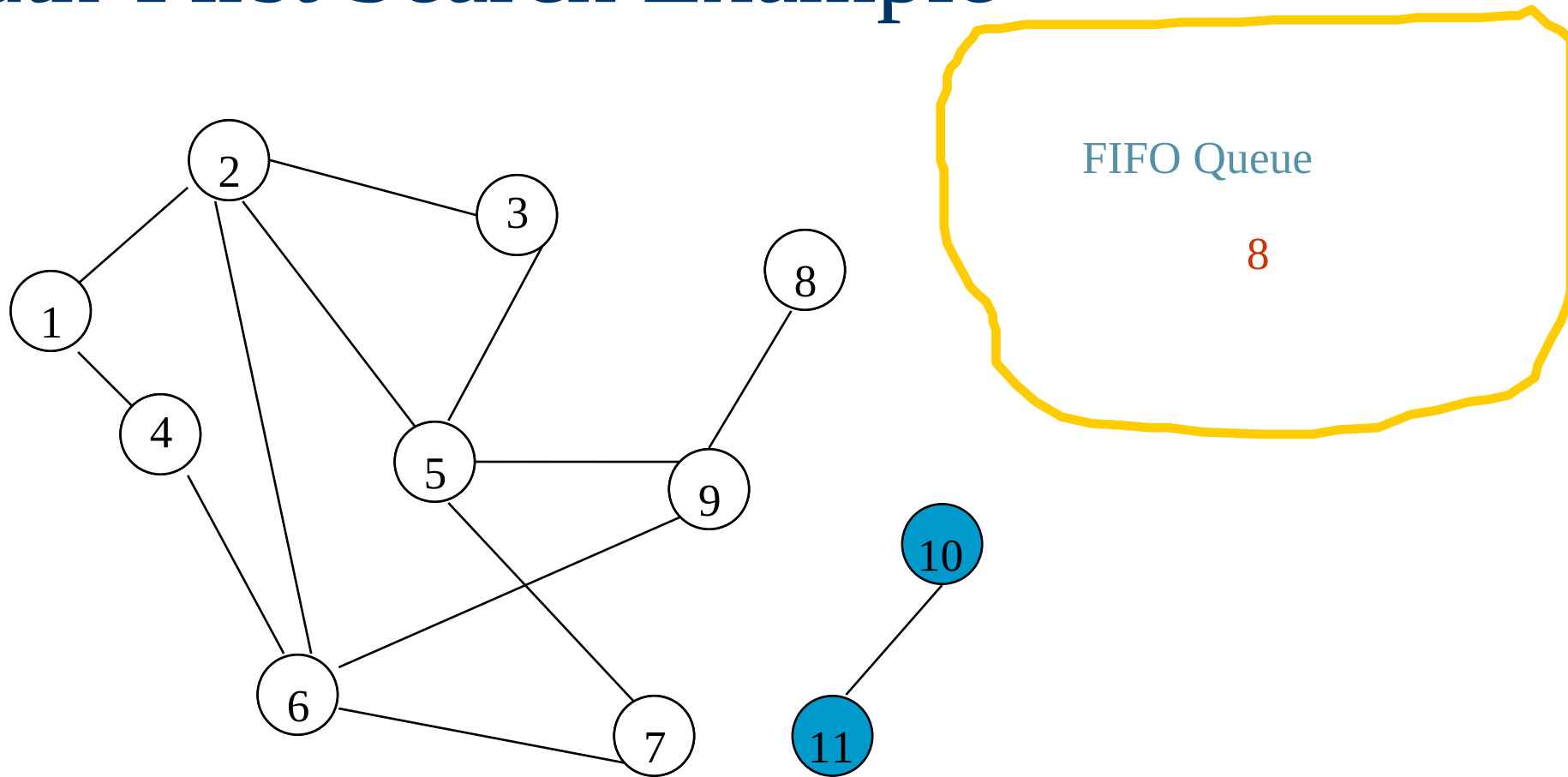
Remove 7 from Q; visit adjacent unvisited vertices;
put in Q.

Breadth-First Search Example



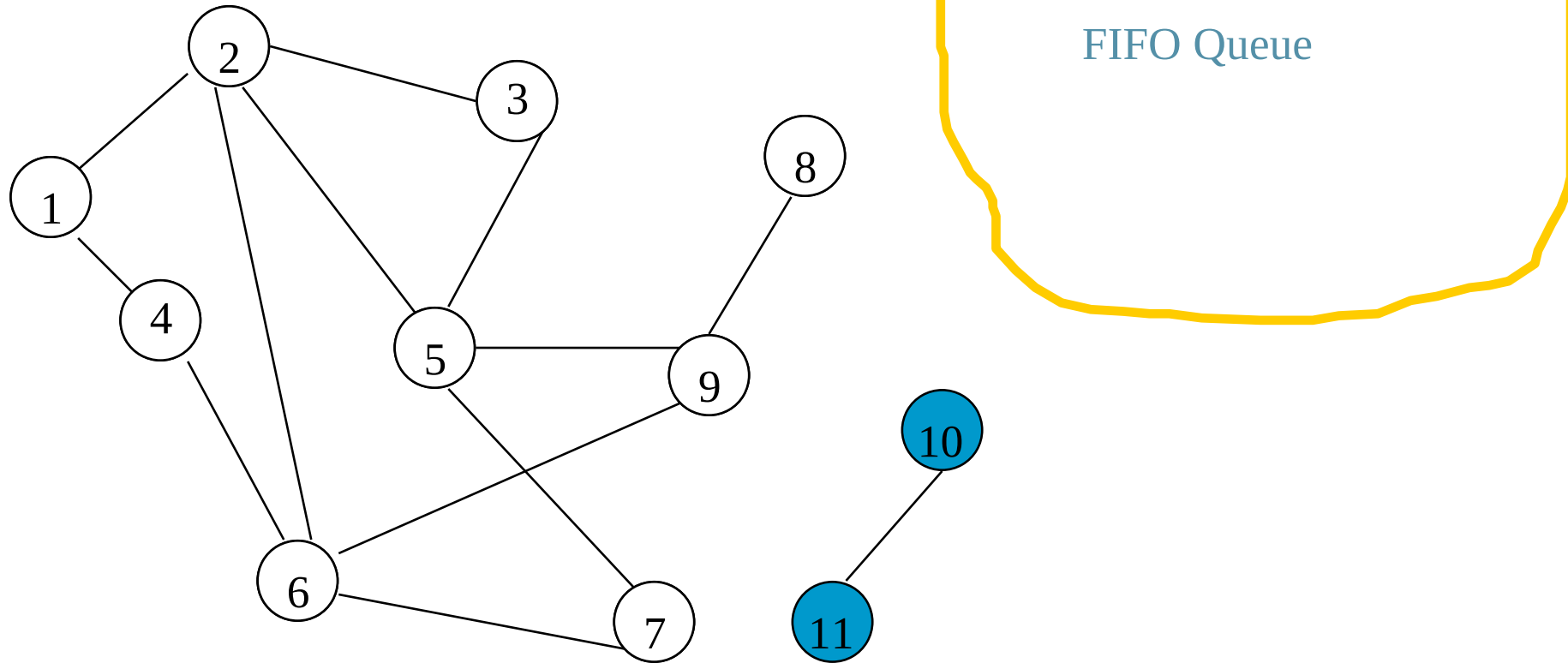
Remove 7 from Q; visit adjacent unvisited vertices;
put in Q.

Breadth-First Search Example



Remove 8 from Q; visit adjacent unvisited vertices;
put in Q.

Breadth-First Search Example



Queue is empty. Search terminates.

Pseudocode for BFS

BFS(v)

```
{ Label vertex v as reached.  
  Initialize Q queue with only v in it  
  while(Q is not empty)  
  {  
    delete a vertex w from the queue;  
    Let u be any vertex adjacent from w;  
    while(u)  
    {  
      if(u has not been labeled){  
        Add u to queue;  
        label u as reached; }  
      u=next vertex adjacent from w;  
    }  
  }  
}
```

BFS(v)

```
{  Label vertex v as reached.          ----- 0(1)
    Initialize Q queue with only v in it ----- 0(1)
    while(Q is not empty)              ----- 0(n)
    {
        delete a vertex w from the queue; ----- 0(n)
        Let u be any vertex adjacent from w;
        while(u)
        {
            if(u has not been labeled){
                Add u to queue;
                label u as reached; }
            u=next vertex adjacent from w;
        }
    }
}
```

For adjacency matrix
representation of
the graph then $O(n)$
time. Total steps
 $O(n^2)$

For adjacency list
representation
 $O(\text{degree}(w))$. Total
steps $O(n + e)$

Time Complexity of BFS: $O(n + e)$, where $n = |V|$ and $e = |E|$

Applications of BFS



- Breadth-first search can be used to solve many problems such as:



- Finding all connected components in a graph G .



- Finding all nodes within an individual connected component.



- Finding the shortest path between two nodes, u and v , of an unweighted graph.

DFS Algorithm

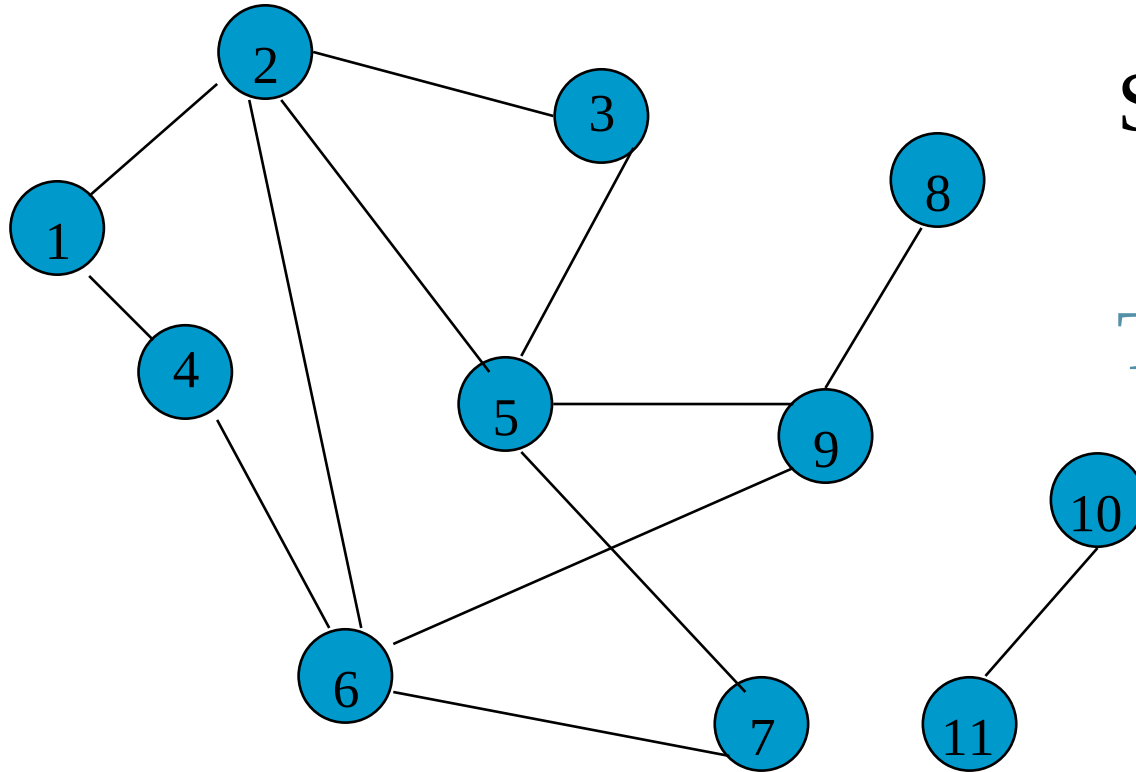
```
DFS (v)
{
    Label v as visited;
    for each verted w adjacent from v
    {
        if (w is not visited)
            DFS(w);
    }
}
```

$O(n)$

**For adjacency matrix
representation of the
graph then $O(n)$ time.
Total steps $O(n^2)$**

**For adjacency list
representation
 $O(\text{degree}(w))$. Total steps
 $O(n + e)$**

Depth-First Search Example



Start search at vertex 1.

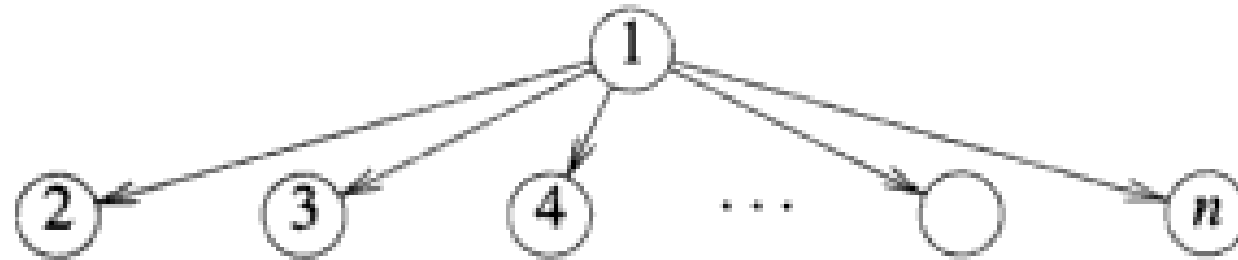
Trace the DFS algorithm

Applications of DFS

- Finding a path between two specified nodes, u and v , of an unweighted graph.
- Finding a path between two specified nodes, u and v , of a weighted graph.
- Finding whether a graph is connected or not.
- Computing the spanning tree of a connected graph.



(a) Worst case for dfs (1) ; best case for bfs (1)



(b) Best case for dfs (1) ; worst case for bfs (1)

Time complexity wise both DFS and BFS takes $O(n+e)$ time. Space complexity wise best and worst case for dfs and bfs has given in the above figure (a) and (b).